第 10 节 实操任务 2：搭建 5 节点工作流

> **学习目标**：`python workflows/graph.py` 能流式输出 5 个节点的执行日志
> 目标文件：`workflows/nodes.py` + `workflows/graph.py`

步骤 1：用 AI 编程工具生成 nodes.py

以下代码可以用 **OpenCode**、**Claude Code**、**Cursor**、**Trae** 或**通义灵码**等任意 AI 编程工具生成。

提示词：

```
请帮我编写 workflows/nodes.py，定义 LangGraph 工作流的 5 个节点函数：

需求：
1. 每个节点是纯函数：接收 KBState，返回 dict（部分状态更新）
2. 节点列表：
   - collect_node: 调用 GitHub Search API 采集 AI 相关仓库（urllib.request）
   - analyze_node: 用 LLM 对每条数据生成中文摘要、标签、评分
   - organize_node: 过滤低分条目(< 0.6)、按 URL 去重、如有审核反馈则用 LLM 修正
   - review_node: LLM 四维度评分（摘要质量/标签准确/分类合理/一致性），iteratior
   - save_node: 将 articles 写入 knowledge/articles/ 目录的 JSON 文件
3. 使用 workflows/model_client.py 的 chat() 和 chat_json() 函数
   - chat(prompt, system=...) 返回 (text, usage)
   - chat_json(prompt, system=...) 返回 (parsed_json, usage)
   - accumulate_usage(tracker, usage) 累加 token 统计
4. 使用 workflows/state.py 的 KBState
5. 每个节点开头打印 [NodeName] 日志

关键设计点：
- review_node 的审核 prompt 要求输出 JSON: {"passed": bool, "overall_score"
- organize_node 在 iteration > 0 且有 feedback 时，调用 LLM 做定向修改
- save_node 同时更新 index.json 索引文件
```

生成的代码：（参考实现，节选关键节点）

```
"""工作流节点定义 — 5 个核心节点

节点调用链:
    collect → analyze → organize → review → (conditional) → save
                                      ↑                        |
                                      └── organize (retry) ←────┘
"""

import json
```

```
import os
from datetime import datetime, timezone

from workflows.model_client import accumulate_usage, chat, chat_json
from workflows.state import KBState


def collect_node(state: KBState) -> dict:
    """采集节点：调用 GitHub Search API 获取 AI 相关仓库"""
    import urllib.request

    today = datetime.now(timezone.utc).strftime("%Y-%m-%d")
    sources: list[dict] = []

    url = "https://api.github.com/search/repositories?q=topic:ai+topic:a
    headers = {"Accept": "application/vnd.github.v3+json"}
    token = os.getenv("GITHUB_TOKEN", "")
    if token:
        headers["Authorization"] = f"token {token}"

    try:
        req = urllib.request.Request(url, headers=headers)
        with urllib.request.urlopen(req, timeout=15) as resp:
            data = json.loads(resp.read().decode())

        for repo in data.get("items", []):
            sources.append({
                "source": "github",
                "title": repo["full_name"],
                "url": repo["html_url"],
                "description": repo.get("description", ""),
                "stars": repo.get("stargazers_count", 0),
                "collected_at": datetime.now(timezone.utc).isoformat(),
            })
    except Exception as e:
        sources.append({"source": "github", "title": "[ERROR]", "descrip

    print(f"[Collector] 采集到 {len(sources)} 条原始数据")
    return {"sources": sources}


def analyze_node(state: KBState) -> dict:
    """分析节点：用 LLM 对每条数据进行深度分析"""
    sources = state["sources"]
    analyses: list[dict] = []
    tracker = state.get("cost_tracker", {})

    for item in sources:
```

```
        if item.get("title", "").startswith("[ERROR]"):
            continue

        prompt = f"""请分析以下技术项目，用 JSON 格式返回：
项目名: {item['title']}
描述: {item.get('description', '无描述')}

返回格式: {{"summary": "200字中文摘要", "tags": ["标签"], "relevance_score":

        try:
            result, usage = chat_json(prompt)
            tracker = accumulate_usage(tracker, usage)
            analyses.append({**item, **result})
        except Exception as e:
            print(f"[Analyzer] 分析失败: {item['title']} - {e}")
            analyses.append({**item, "summary": f"分析失败: {e}", "relevan

    print(f"[Analyzer] 完成 {len(analyses)} 条分析")
    return {"analyses": analyses, "cost_tracker": tracker}


def organize_node(state: KBState) -> dict:
    """整理节点：过滤 + 去重 + 如有反馈则修正"""
    analyses = state["analyses"]
    feedback = state.get("review_feedback", "")
    iteration = state.get("iteration", 0)
    tracker = state.get("cost_tracker", {})

    # 过滤低分条目
    qualified = [a for a in analyses if a.get("relevance_score", 0) >= 0

    # URL 去重
    seen, unique = set(), []
    for item in qualified:
        url = item.get("url", "")
        if url and url not in seen:
            seen.add(url)
            unique.append(item)

    # 如果有审核反馈，用 LLM 修正
    if feedback and iteration > 0:
        prompt = f"""你是知识库编辑。请据以下反馈改进条目。
反馈: {feedback}
条目: {json.dumps(unique, ensure_ascii=False)}
返回改进后的 JSON 数组。"""

        try:
            improved, usage = chat_json(prompt)
```

```
            tracker = accumulate_usage(tracker, usage)
            if isinstance(improved, list):
                unique = improved
        except Exception:
            pass

    # 格式化为标准条目
    today = datetime.now(timezone.utc).strftime("%Y-%m-%d")
    articles = [
        {"id": f"{today}-{i:03d}", **{k: item.get(k, "") for k in ["titl
        for i, item in enumerate(unique)
    ]

    print(f"[Organizer] 整理出 {len(articles)} 条知识条目 (迭代 {iteration}
    return {"articles": articles, "cost_tracker": tracker}


def review_node(state: KBState) -> dict:
    """审核节点：LLM 四维度评分"""
    articles = state.get("articles", [])
    iteration = state.get("iteration", 0)
    tracker = state.get("cost_tracker", {})

    if not articles:
        return {"review_passed": True, "review_feedback": "没有条目需要审核

    prompt = f"""你是知识库质量审核员。请审核以下条目：
{json.dumps(articles[:5], ensure_ascii=False, indent=2)}

评分维度（1-5分）：摘要质量、标签准确性、分类合理性、整体一致性
overall_score >= 3.5 即通过。第 {iteration + 1} 次审核（最多3次）。

返回 JSON: {{"passed": true/false, "overall_score": 4.2, "feedback": "改进

    try:
        result, usage = chat_json(prompt, system="你是严格但公正的知识库审核
        tracker = accumulate_usage(tracker, usage)
        passed = result.get("passed", False)
        feedback = result.get("feedback", "")

        if iteration >= 2:
            passed = True
            feedback += "\n[系统] 已达最大审核次数(3次)，强制通过。"

        print(f"[Reviewer] 得分: {result.get('overall_score', '?')}, 通过
    except Exception as e:
        passed, feedback = True, f"审核失败: {e}，自动通过"

```

```
    return {"review_passed": passed, "review_feedback": feedback, "itera


def save_node(state: KBState) -> dict:
    """保存节点：写入 JSON 文件 + 更新索引"""
    articles = state.get("articles", [])
    if not articles:
        print("[Saver] 没有条目需要保存")
        return state

    base_dir = os.path.dirname(os.path.dirname(os.path.abspath(__file__)
    articles_dir = os.path.join(base_dir, "knowledge", "articles")
    os.makedirs(articles_dir, exist_ok=True)

    for article in articles:
        filepath = os.path.join(articles_dir, f"{article['id']}.json")
        with open(filepath, "w", encoding="utf-8") as f:
            json.dump(article, f, ensure_ascii=False, indent=2)

    # 更新索引
    index_path = os.path.join(articles_dir, "index.json")
    index = json.load(open(index_path, encoding="utf-8")) if os.path.exi
    existing_ids = {e["id"] for e in index}
    for a in articles:
        if a["id"] not in existing_ids:
            index.append({"id": a["id"], "title": a.get("title", ""), "c
    with open(index_path, "w", encoding="utf-8") as f:
        json.dump(index, f, ensure_ascii=False, indent=2)

    print(f"[Saver] 保存 {len(articles)} 篇文章")
    return state
```

步骤 2：用 AI 编程工具生成 graph.py

提示词：

```
请帮我编写 workflows/graph.py，组装 LangGraph 工作流：

需求：
1. 使用 langgraph.graph 的 StateGraph, END
2. 导入 workflows/nodes.py 的 5 个节点函数
3. 导入 workflows/state.py 的 KBState
4. 线性边: collect → analyze → organize → review
5. 条件边: review 之后根据 review_passed 分支
   - True → save → END
   - False → organize (回到整理节点修正)
6. 入口点: collect
```

```
7. build_graph() 函数返回编译后的 app
8. if __name__ == "__main__" 流式执行并打印每个节点的关键输出

注意：使用真实的 LangGraph API：
- StateGraph(KBState)
- graph.add_node("name", function)
- graph.add_edge("a", "b")
- graph.add_conditional_edges("source", router_fn, {"key": "target"})
- graph.set_entry_point("name")
- graph.add_edge("save", END)
- graph.compile()
```

生成的代码：（参考实现）

```
"""LangGraph 工作流图定义 — 采集-分析-审核流水线

工作流拓扑:
    collect → analyze → organize → review ──→ save (通过)
                            ↑                    │
                            └────────────────────┘ (未通过)
"""

from langgraph.graph import END, StateGraph

from workflows.nodes import (
    analyze_node, collect_node, organize_node, review_node, save_node,
)
from workflows.state import KBState


def should_continue(state: KBState) -> str:
    """条件路由：审核通过 → save，未通过 → organize"""
    if state.get("review_passed", False):
        return "save"
    return "organize"


def build_graph():
    """构建知识库 V3 工作流"""
    graph = StateGraph(KBState)

    # 添加节点
    graph.add_node("collect", collect_node)
    graph.add_node("analyze", analyze_node)
    graph.add_node("organize", organize_node)
    graph.add_node("review", review_node)
    graph.add_node("save", save_node)

    # 线性边
```

```
    graph.add_edge("collect", "analyze")
    graph.add_edge("analyze", "organize")
    graph.add_edge("organize", "review")

    # 条件边：审核循环
    graph.add_conditional_edges(
        "review", should_continue, {"save": "save", "organize": "organiz
    )

    # 终点
    graph.add_edge("save", END)

    # 入口
    graph.set_entry_point("collect")

    return graph


app = build_graph().compile()


if __name__ == "__main__":
    print("=" * 60)
    print("AI 知识库 — LangGraph 工作流启动")
    print("=" * 60)

    initial_state: KBState = {
        "sources": [], "analyses": [], "articles": [],
        "review_feedback": "", "review_passed": False,
        "iteration": 0, "cost_tracker": {},
    }

    for event in app.stream(initial_state):
        node_name = list(event.keys())[0]
        print(f"\n--- [{node_name}] 完成 ---")

    print("\n" + "=" * 60)
    print("工作流执行完毕")
```

步骤 3：理解代码

> `请解释 workflows/graph.py 的设计：`
>
> `1. add_conditional_edges 的三个参数分别是什么意思？`
>
> `2. should_continue 返回的字符串和第三个参数的字典有什么关系？`
>
> `3. review 未通过时回到 organize，organize 怎么知道要修正？`

> `4. 循环什么时候结束？`

步骤 4：运行测试

```
cd ~/ai-knowledge-base
python workflows/graph.py
```

期望输出：

```
============================================================
AI 知识库 — LangGraph 工作流启动
============================================================
[Collector] 采集到 10 条原始数据

--- [collect] 完成 ---
[Analyzer] 完成 8 条分析

--- [analyze] 完成 ---
[Organizer] 整理出 6 条知识条目 (迭代 0)

--- [organize] 完成 ---
[Reviewer] 得分: 3.8, 通过: True (迭代 1/3)

--- [review] 完成 ---
[Saver] 保存 6 篇文章

--- [save] 完成 ---
============================================================
工作流执行完毕
```

步骤 5：提交到 Git

```
git add workflows/nodes.py workflows/graph.py
git commit -m "feat: add 5-node LangGraph workflow with review loop"
```

完成！5 个节点的 LangGraph 工作流已经能跑了。下一步加条件边测试审核循环。